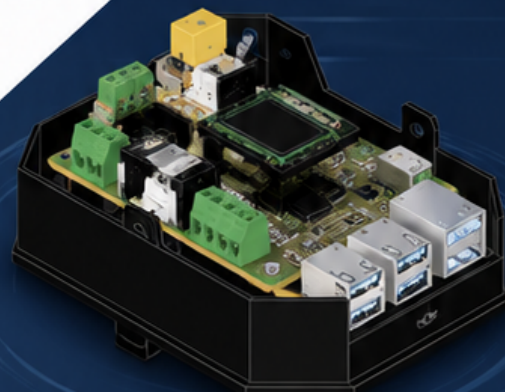


Configurable remote lab interface

user guide



1. Configure

Select devices, add controls and parameters



2. Save

Save your configuration as a custom setup



3. Launch

Your remote lab interface is ready to use



4. Experiment

Run experiments, view data and control in real time

Contents

1	Introduction	2
1.1	Pre-requisites	2
1.1.1	Making your experiment available online	2
1.1.2	Video stream	3
1.1.3	Firmware and microcontroller	3
2	Configuring control commands	4
2.1	Commands with fixed values	4
2.2	Commands with variable values	4
2.3	Saving, loading, updating and deleting	7
3	Displaying received data	8
3.1	Viewing raw JSON messages	8
3.2	Recording data snapshots	8
3.3	Nested messages and data types	8
3.4	Recording data continuously	10
3.5	Downloading data	10
4	Graphing data	11
4.1	Plotting functions	11
5	Example: duty cycle	12

Version

User Guide Version: 1.1.0

Config UI version: 1.0.2

Updated on: 12/08/2026

Originally published: 01/06/2026

Author: David P. Reid

Cover image is a manually modified version of an image generated by ChatGPT.

1 Introduction

This is a user guide for the configurable user interface for practable remote laboratories. It provides instructions on how to setup the configurable interface for your connected remote lab using examples from pre-existing remote laboratories. This guide assumes you have existing hardware to interact with - via a pre-configured Portal or an existing remote laboratory - but see the pre-requisites below for details.

Visit <https://practable.io> for details of the practable remote laboratory system and available experiments. Code and documentation for existing remote labs and for self-hosting a practable relay server, are available on GitHub at <https://github.com/practable>.

The configurable user interface contains four default components: 1) video feed, 2) graph, 3) data table and 4) control panel. Figure 1 shows the default user interface before configuration.

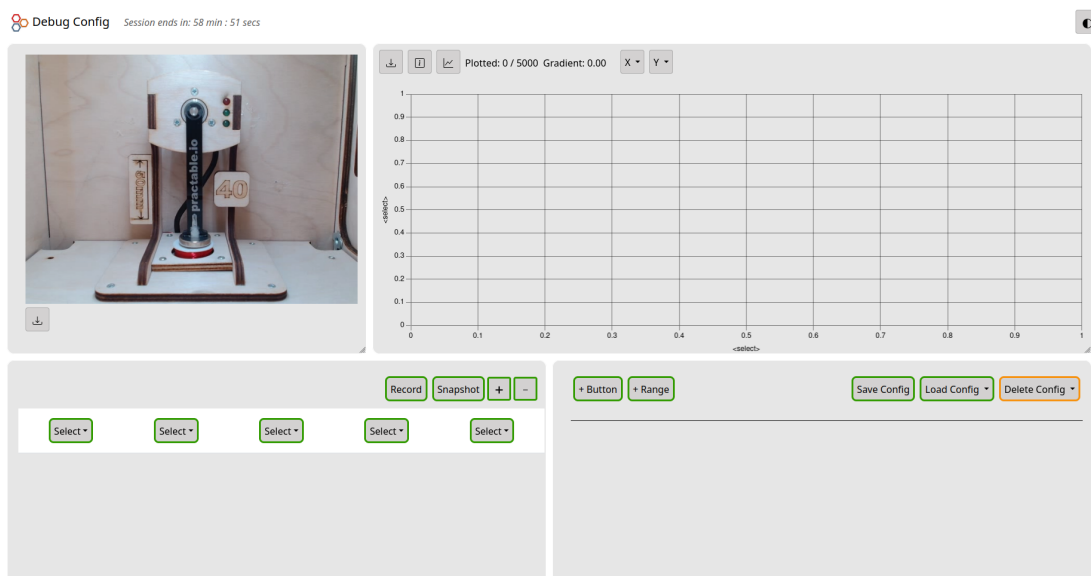


Figure 1: Screenshot of the configurable user interface on page load, before configuration, showing video feed (top left), graph (top right), data table (bottom left) and control panel (bottom right). This interface is connected to the pendulum remote laboratory.

These instructions will use the pendulum remote laboratory as an example of configuring control of remote hardware and displaying received data. Documentation and firmware for the pendulum remote lab can be found here: <https://github.com/practable/penduino>

1.1 Pre-requisites

To use the configurable interface to interact with a practable remote laboratory requires i) a link to hardware that is accessible online, ii) a video stream from a connected webcam and iii) firmware that sends and receives messages in JSON format. See below for details.

1.1.1 Making your experiment available online

It is necessary to have hardware/microcontroller/video that is already available online through a practable remote lab server. The recommended means of achieving this is to use a pre-configured Portal (Figure 2) to which you connect a webcam and microcontroller, with the Portal taking care of connection and streaming of video, data/control commands. Portals are available by getting in touch with the practable team. A user guide for the Portal is available at <https://github.com/practable/practable-portal>. If instead you wish to host your own server and client code for streaming video and data, then please see the details on GitHub at <https://github.com/practable>.

If you have a pre-configured Portal device, then you should also have been provided with a link to access the booking page for the device. This link will include the configurable user interface by default.

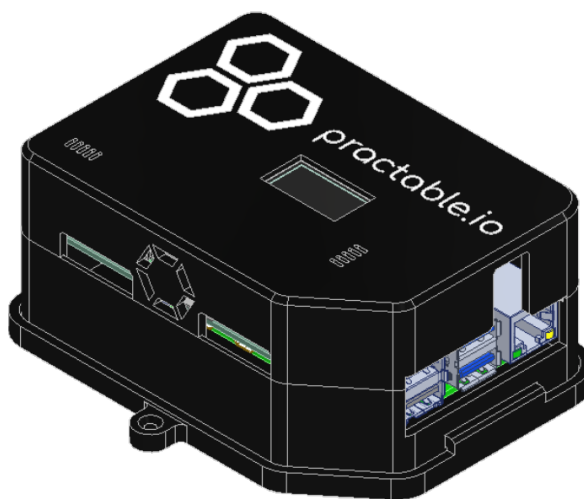


Figure 2: Image of the practable Portal for pre-configured connection of a remote laboratory.

1.1.2 Video stream

In its current form, the configurable interface expects to connect and display a video stream from your remote laboratory. This can most simply be setup by connecting a webcam (we recommend and use the Logitech C920) to a pre-configured Portal (see Section 1.1.1). A version of the configurable interface that does not include a video feed is planned for a future release.

1.1.3 Firmware and microcontroller

The configurable interface can send control commands to hardware and receive data from hardware via a microcontroller attached (via USB) to the Portal (or existing remote lab). We typically use and recommend Arduino (Nano, Every, IoT33) or ESP32 microcontrollers.

For proper functioning of the configurable interface, the firmware running on the microcontroller should send and receive commands in JSON (JavaScript Object Notation) format. An example of a simple JSON message sent by the Pendulum remote lab is:

```
{"enc": 0, "time": 3632631459}
```

The configurable user interface can parse JSON messages to extract key:value pairs of many different types, including values that are themselves JSON objects (see Section 3.3).

2 Configuring control commands

This section describes how to setup UI components to send JSON commands to your remote experiment. Combinations of buttons and range sliders can be created to support all required control commands and variables and your configured interfaces can be saved for later use.

Control commands are sent to practable remote labs as JSON strings. The content of these strings can be completely customised for your remote lab, as long as they remain valid JSON. Configuring the interface for your remote experiment involves defining commands that are then attached to buttons for easy, repeatable execution of those commands. Some commands may use a fixed JSON string containing key:value pairs that never change, while others may include values that update based on a changing parameter. The following will describe how to configure each type of command.

2.1 Commands with fixed values

Commands with fixed key:value pairs do not include any values based on an updatable variable. This may include commands that update the state in which firmware is running. As an example, a start command may be of the form:

```
{"cmd":"start"}
```

where the same fixed JSON string is always sent to enter a 'start' state. The procedure below will allow you to attach such a command to a button on the interface:

- 1) Click [+ **Button**]. Input fields and buttons should appear as shown in Figure 3a.
- 2) In the input with '**Button name**' as a placeholder type the name of your command (this is the string that will display on the button when it is created).
- 3) In the input with the '**JSON Command**' placeholder, type the exact JSON command that should be sent when this button is clicked. Figure 3b gives an example.
- 4) Since this is a fixed string command with no variable included, click [**Create**] to display the new button, while leaving the [**Inc. Params**] checkbox **unselected**. Clicking on this new button should now send the corresponding JSON command.

Repeat this procedure for all the fixed commands required to control your hardware. Ensure that each button has a unique name. See Section 2.3 for how to save your configured UI, load a previously saved configuration and delete buttons and configurations.

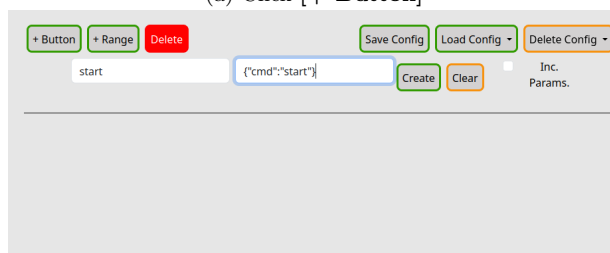
2.2 Commands with variable values

Some commands will include variable values that can be changed by a user, with the command expected to send whatever the current value has been set to. In these cases the JSON command is setup with templated values that will point to a variable, rather than a single fixed string. For example, sending a drive command to the pendulum includes a value for the "strength" of the driving force and this value can be updated to change the amplitude of the pendulum oscillations. To configure a button to send such a variable value we must add both a button for sending the JSON command and a range input to store the variable to be updated.

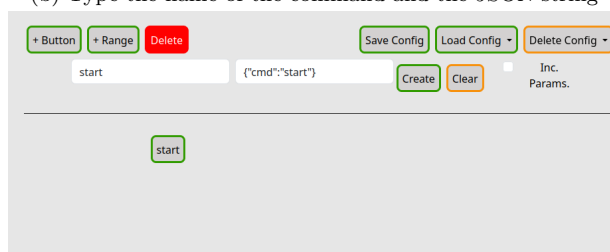
- 1) Click [+ **Button**].
- 2) Click on the checkbox toggle [**Inc. Params**] to let the UI know that you wish to add a variable parameter to this command. You should see additional input fields as shown in Figure 4a.
- 3) In the input with '**Button name**' as a placeholder type the name of your command (this is the string that will display on the button when it is created).
- 4) In the input with the '**JSON Command**' placeholder, type the JSON command that should be sent when this button is clicked. Where the variable should be included add the name of that variable between



(a) Click [+ **Button**]



(b) Type the name of the command and the JSON string



(c) Click [**Create**]. Your button with that command will display.

Figure 3: Screenshots showing how to attach a fixed command to a button.

angle brackets ('< >'). For example, Figure 3b shows a **drive** parameter included in the JSON string as **<drive>**.

5) In the input with the '**Param1**' placeholder, type the name of the parameter, exactly as it is written in the JSON string but **without** the angle brackets ('< >')

6) Click [**Create**] to display the new button (see Figure 4c).

7) Click [+ **Range**] to add a range slider that creates the variable we wish to send with our command (e.g. drive in the example). Figure 4d shows the input fields that show after clicking [+ Range].

8) In the '**Parameter name**' field add the name of the variable you want to attach this range to - exactly as written in the JSON command and '**Param1**' fields above. In our example we would write **drive** into this input field.

9) In the remaining inputs add the required minimum and maximum values for the range slider and a step size. Click [**Create**] and your new parameter should display as a range slider like that shown in Figure 4e.

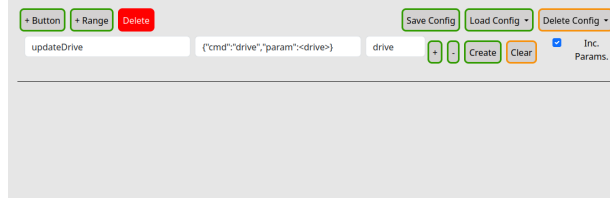
After following the above steps, you should now be able to change the variable value using the slider and then when a command is sent, by clicking the corresponding button, it will send the current value of that variable. To confirm that the setup has been performed correctly, you can use the browser console. After updating the variable and clicking the command button, the console will display the sent command which can be checked.

If a command requires more than one variable parameter, follow the above procedure, but after step 2 click the [+] button to add a second parameter, duplicating the steps for parameter 1 for this second parameter and adding a second range slider for control of this second variable. Repeat for additional variables.

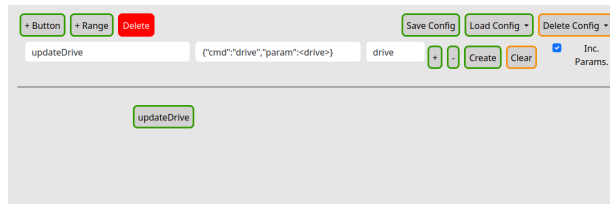
Variable commands and parameters can be updated and deleted. See section 2.3 for details on how to do this.



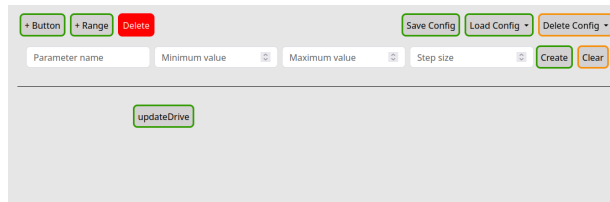
(a) Click **[+ Button]** and toggle the **[Inc. Params]** checkbox.



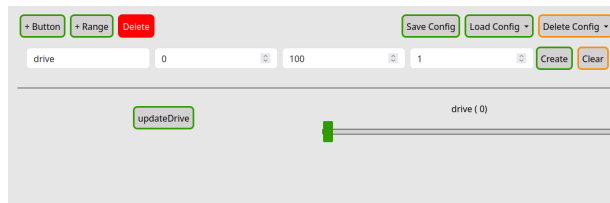
(b) Type the name of the command and the JSON string, including the parameter between angle brackets.



(c) Click **[Create]**. Your button with that command will display.



(d) Click **[+ Range]**. You should see the input fields shown here.



(e) Type the appropriate values and click **[Create]**. A range slider with the new variable will display.

Figure 4: Screenshots showing how to attach a variable command to a button.

2.3 Saving, loading, updating and deleting

You can **save** your configured interface at any time by clicking [**Save Config**]. Type the name you want to give to this configuration in the ‘**config name**’ input and then click the [✓] button to save. This saves your configuration in browser localStorage - this means that the saved configuration is only available on your current device and browser. No auto-save functionality currently exists, so you should manually save whenever you are happy that a function has been successfully added to the UI.

Any previously saved configurations can be **loaded** by clicking [**Load Config**] and selecting from the dropdown menu of your previously saved configurations. If a configuration is no longer required, it can be deleted permanently by clicking [**Delete Config**] and selecting from the dropdown.

Individual buttons and range sliders can be deleted from a configuration by clicking the red [**Delete**] button, which will highlight all the deletable objects in red, and then clicking the component you wish to delete. If after clicking [Delete] you decide you do not wish to delete anything, click [**Cancel**].

Buttons and ranges can be **updated** rather than deleted by creating a new button or range with the same name. This will replace the previous element with that name for the new one. Alternatively, the button or range can be first deleted and then recreated.

3 Displaying received data

The interface will automatically parse the JSON messages received and make the data contained within those messages available for display, recording, graphing and download. While the available data is based on the content of received JSON messages, you need to configure which data to display and record, details of which can be found in the sections below.

3.1 Viewing raw JSON messages

The raw JSON formatted messages received by the configurable user interface are displayed automatically and are shown in a component title **Received Messages** that is visible by scrolling down the browser window. The ten most recent messages are displayed. If no messages are displayed then this could be because there is a problem with the data connection or that the received messages are not in valid JSON format. The browser tools could be used to confirm whether messages (of any format) are being received through the data websocket. Figure 5 gives an example screenshot of simple data received from the pendulum remote lab.

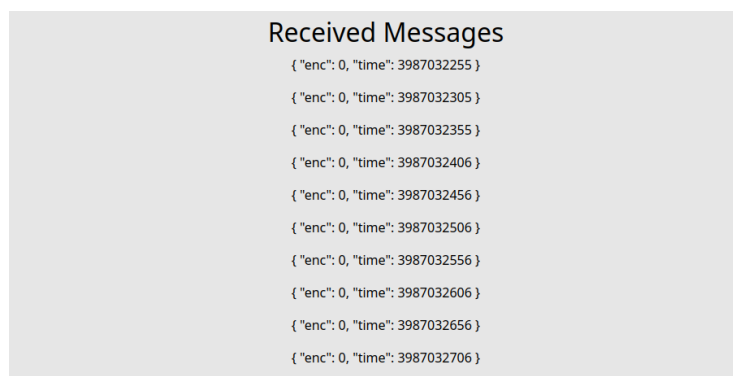


Figure 5: Screenshot of the component showing the 10 latest raw JSON formatted messages received from the (pendulum) hardware.

3.2 Recording data snapshots

Rather than viewing the received messages as raw JSON strings, the data display component can extract values and display them in a table (which is also used for graphing, see Section 4).

After clicking on a **[Select]** dropdown, a list of the parameters present in the received JSON messages is shown. Selecting a variable will rename the dropdown with the selected parameter and display the most recently received value of that parameter. This value should update as new messages are received.

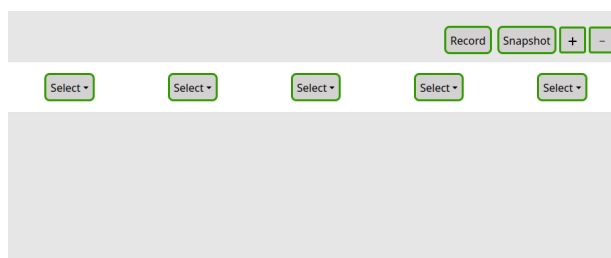
By default, five dropdowns are visible, but the number of parameters you can display can be changed by clicking the **[+]** or **[-]** buttons. See Figure 6c where three dropdowns have been removed by clicking **[-]** three times, to leave the two variables of interest for the pendulum experiment.

To record a snapshot of the current data, click **[Snapshot]**. This will add a new row to your table with the recorded data. Figure 6d shows the data table after three snapshots have been taken. Data can be cleared by clicking **[Clear]**.

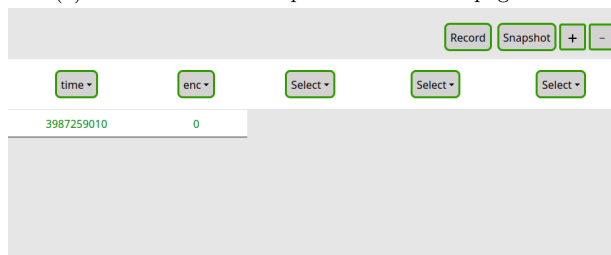
3.3 Nested messages and data types

The interface does not expect data of any particular type, other than the requirement for messages to be valid JSON. If you select a specific key:value to display, the data table will attempt to display the latest value received and will display values that can be printed as strings, e.g. floats, integers, booleans and strings.

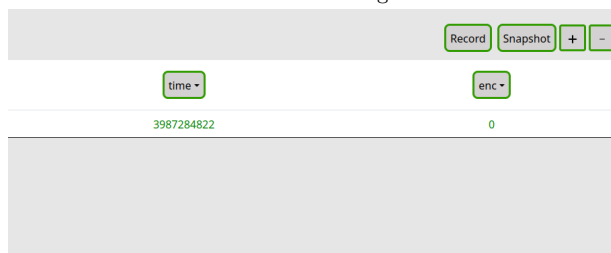
If a value is received as an array of items, then the current value displayed will be the last item in that array. When a snapshot is taken all items in the array will be recorded and displayed in the table.



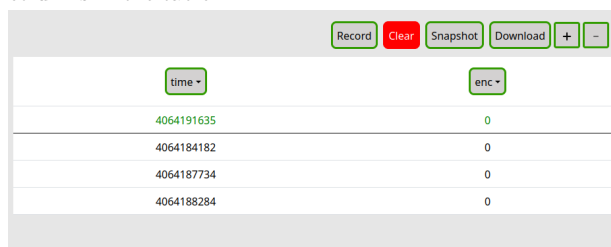
(a) The blank data component as seen on page load.



(b) Clicking on the dropdowns allows you to select variables available in the received JSON messages.



(c) Clicking on the + or - buttons allows you to add or remove columns in the table.



(d) Clicking **Snapshot** will record the data at that moment and add it to the table.

Figure 6: Screenshots showing how to record data snapshots.

If a received JSON message consists of nested JSON objects, that is, if the value of a key:value pair is itself a JSON object with its own key:value pairs, then the interface attempts to extract all the lowest level values that can be displayed. When attempting to select a nested variable to record data it will be displayed in terms of all the keys necessary to extract it. For example, the following received JSON message (which has been formatted to better show the nested structure of the received string):

```
{ "timestamp": 1666047267,
  "payload":
  {
    "state": "STATE_WAIT",
    "servo": 0,
    "encode":
    {
      "pos": [ 0 ],
      "angle": [ " 0.000" ]
    },
    "loadcell":
```

```
{
  "force": [ " -2.399" ]
},
"meta":
{
  "samples": 1, "time": [ 1666046466 ]
}
}
```

would lead to a dropdown menu that looks like the screenshot in Figure 7. To display the latest force value from the received JSON message, you would select the dropdown item **payload.loadcell.force**.

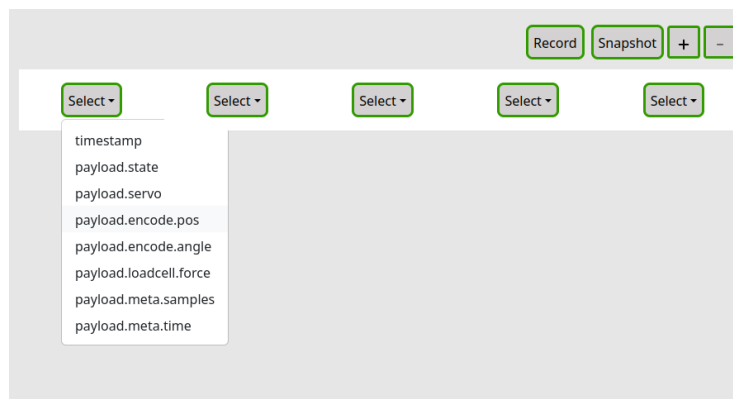


Figure 7: Screenshot showing the datasetselect dropdown for a received JSON message with a nested structure.

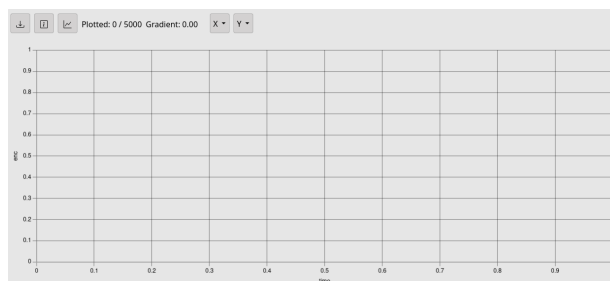
3.4 Recording data continuously

As an alternative to manually recording snapshots, by clicking [**Record**] you can automatically record snapshots whenever a new message is received by the user interface. This will automatically record only the data selected in the dropdowns within this component. Click [**Stop**] to stop recording data and [**Clear**] to delete recorded data.

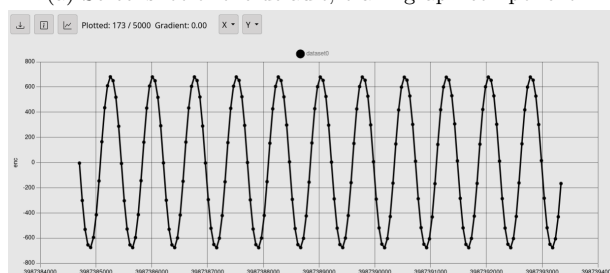
If messages are being received at a high rate then this could potentially overwhelm the browser. We recommend at least 20ms between received messages. Firmware can record at a higher rate and send arrays of data if higher resolution is necessary. There is also a limit of 5000 data points for graphing to maintain good browser performance.

3.5 Downloading data

Recorded data can be downloaded as a comma separated variable (CSV) file and saved to your local computer. After some data has been recorded (either continuously or as manual snapshots) a [**Download**] button will appear at the top of the data table component. Clicking this button will download a CSV for each dataset you have recorded. These will appear in your local Downloads folder (or wherever you have configured your browser to store downloaded files).



(a) Screenshot of the default, blank graph component.



(b) Screenshot of the graph component after selecting and recording data.

Figure 8: Screenshots of the graph component.

4 Graphing data

Recorded data can be automatically plotted in the user interface. The graph component will only plot data that has been recorded in the data table component: either using manual snapshots (Section 3.2) or automatic recording (Section 3.4).

To plot data in the graph component, select the X and Y parameters using the dropdown menus at the top of the component - see Figure 8a. If data is already recorded, then the graph will refresh to show all recorded data. If the graph axes are selected before recording, then the graph will automatically update as data is received.

Information on the graphing component is available in the user interface by clicking the information button [i] in the top left of the graph component.

4.1 Plotting functions

Basic function plotting is available in the user interface graph component by clicking on the button to the right of the information button at the top left of the graph component. A screenshot of the function plotter is shown in Figure 9.

Function plotting is best used after a data set has been recorded. Plotted functions will automatically clear as soon as new data is recorded.

The available functions are linear, quadratic, trigonometric and exponential. The equation for each option and input fields for corresponding parameters are displayed when a function is selected from the dropdown.

After parameters are input, click [Plot] to add the function to the graph component. Click [Clear] to remove the function from the plot. As noted above, the function will automatically clear if new data is recorded.

The function plotter uses the **x step** parameter, the range of values on the x axis of the graph and your equation parameters to calculate and plot y. The **x step** parameter should be updated to plot an appropriate number of data points. If there is a very large range of values on the x axis, then x step should be increased so as not to plot too many points. Inversely, if the x axis range is small you want to reduce the x step so that sufficient data points are plotted. Too many data points may slow your browser down. Inappropriate (e.g. non-numerical) values on the x axis may lead to unexpected plotting behaviour, so function plotting is typically recommended only when simple numerical data has been recorded.

Plot Function
✕

linear ▾

$y = ax + b$

a =

0

▾

b =

0

▾

x step =

0.01

▾

Plot

Clear

Close

Figure 9: Screenshot of the function plotting popup available in the graph component.

5 Example: duty cycle

Example firmware that can be used to test the portal and configurable user interface can be found here: <https://github.com/practable/practable-portal/tree/main/demo>. This firmware has been tested on an Arduino Nano IoT 33.

This firmware outputs a square wave with a duty cycle that can be updated (between 0 - 1) by sending a single command. To configure a button to send the appropriate command with a variable value, follow the procedure in Section 2.2. In this case the ‘JSON command’ structure is:

```
{"set": "duty_cycle", "to": <duty>}
```

and the ‘Param1’ field should be ‘duty’ to match the string in the command representing the variable to include. When adding the parameter as a range, the ‘Parameter name’ should also be **duty** to match the string included in the command above. The min value could be set to 0 and the max to 1, with a step of your choice (e.g. 0.1).

To record and view data being received from the firmware follow the procedures described in Section 3. In this example, the received JSON messages have the following structure:

```
{"timestamp": 923423, "output": 0}
```

and therefore two variables are available to record and display: timestamp and output. To record these values, first select both variables in the data recorder using the ‘Select’ dropdowns as described in Section 3.2. Data can then be recorded and displayed as described in Section 3.